



GitHub

Recommendation System

*Improving Open-Source Project Recommendations
A Hybrid Model Using NCF and XGBoost Ranking*

Jasmine Yang
Chintan Armit

```
... object to mirror_mod.mirror_object
operation == "MIRROR_X":
    mirror_mod.use_x = True
    mirror_mod.use_y = False
    mirror_mod.use_z = False
operation == "MIRROR_Y":
    mirror_mod.use_x = False
    mirror_mod.use_y = True
    mirror_mod.use_z = False
operation == "MIRROR_Z":
    mirror_mod.use_x = False
    mirror_mod.use_y = False
    mirror_mod.use_z = True

#selection at the end -add
mirror_ob.select= 1
modifier_ob.select=1
context.scene.objects.active
("Selected" + str(modifier_ob.name))
mirror_ob.select = 0
= bpy.context.selected_objects
data.objects[one.name].select

print("please select exactly one object")

--- OPERATOR CLASSES ---

... operator):
```

Motivation

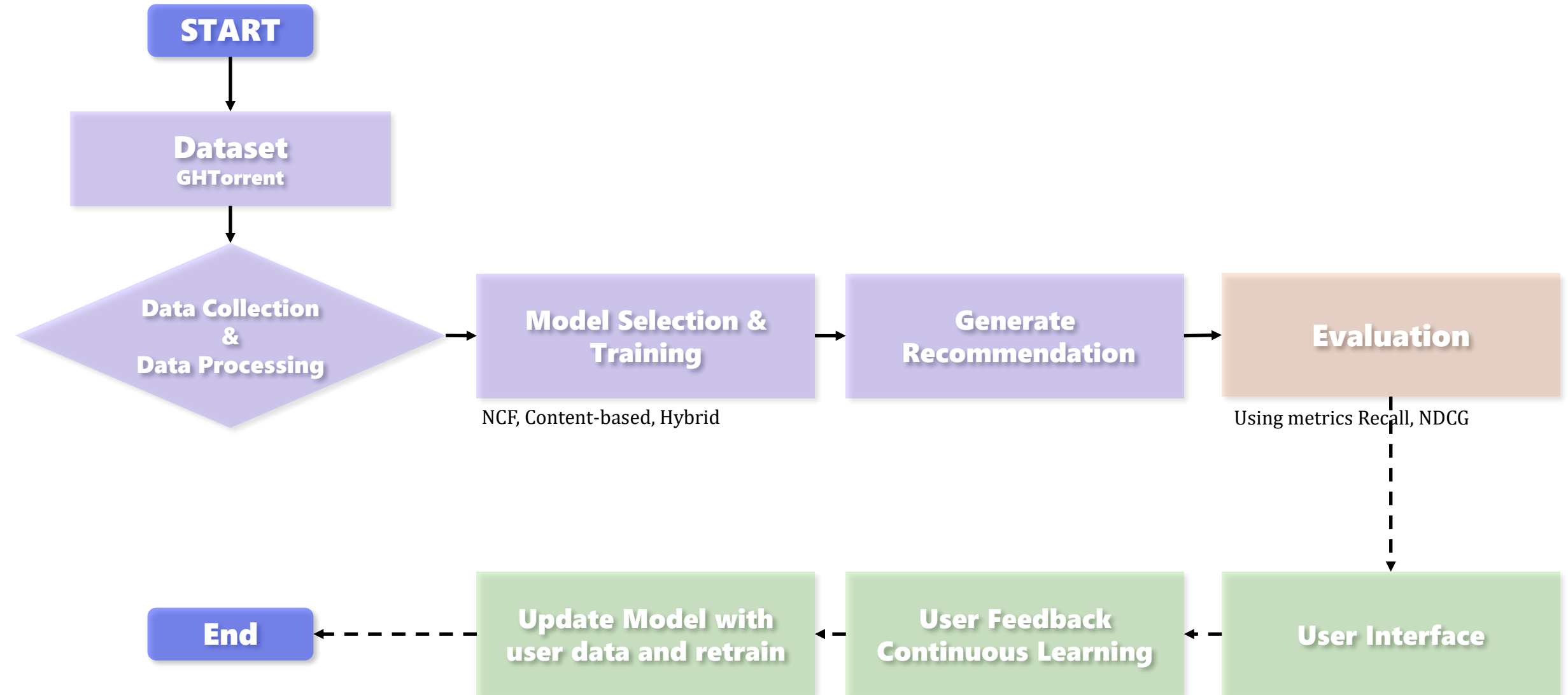
Long Time Contributors	Non-Long-Time Contributors
2.19%	97.81%

[1]

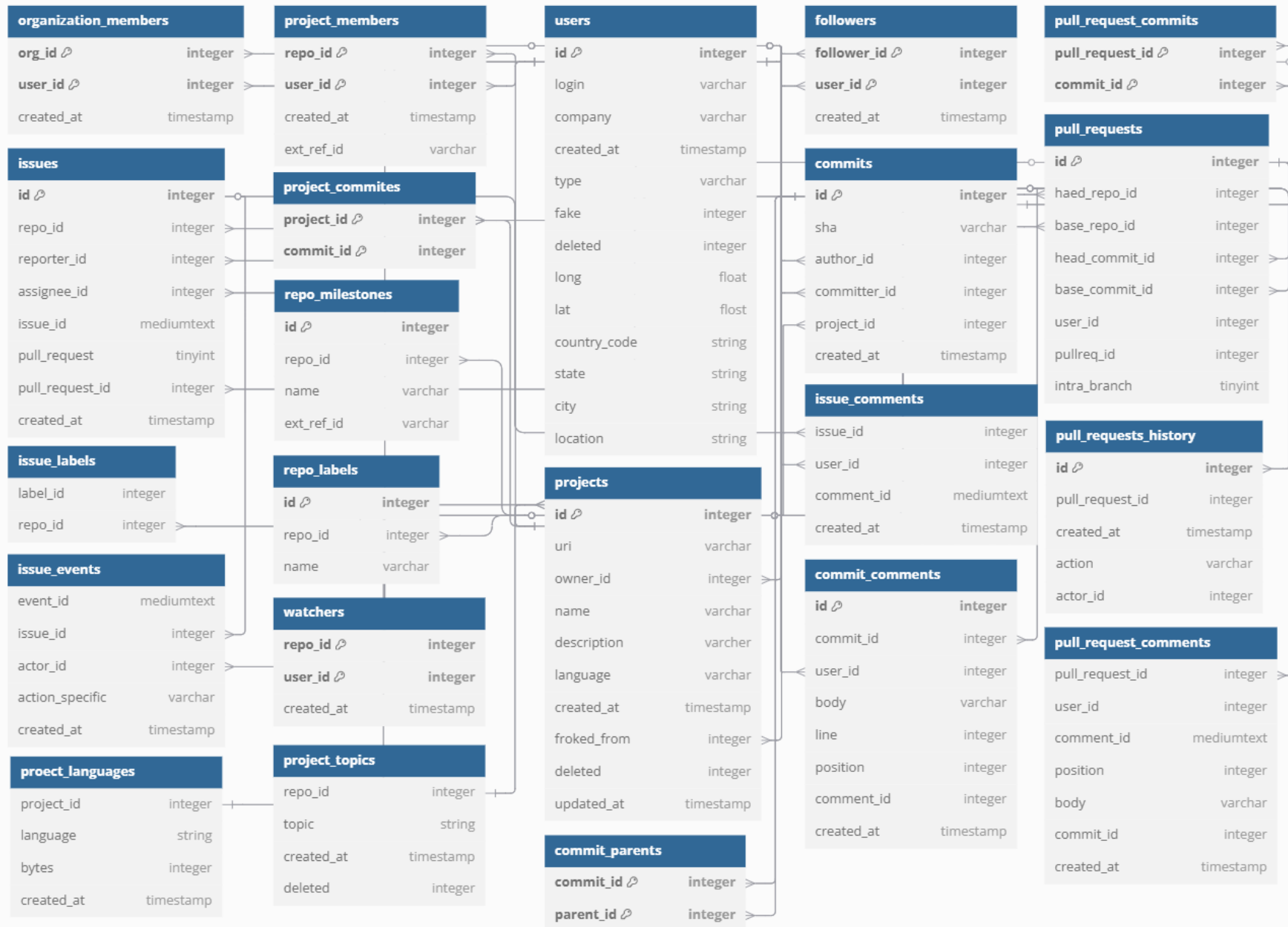
GOAL 💡

Our recommendation system is to **recommend relevant GitHub projects to a specific developer** based on that developer's past interactions, profile, and other contextual metadata, features

Basic flow of Recommender Systems



GH Torrent [2]

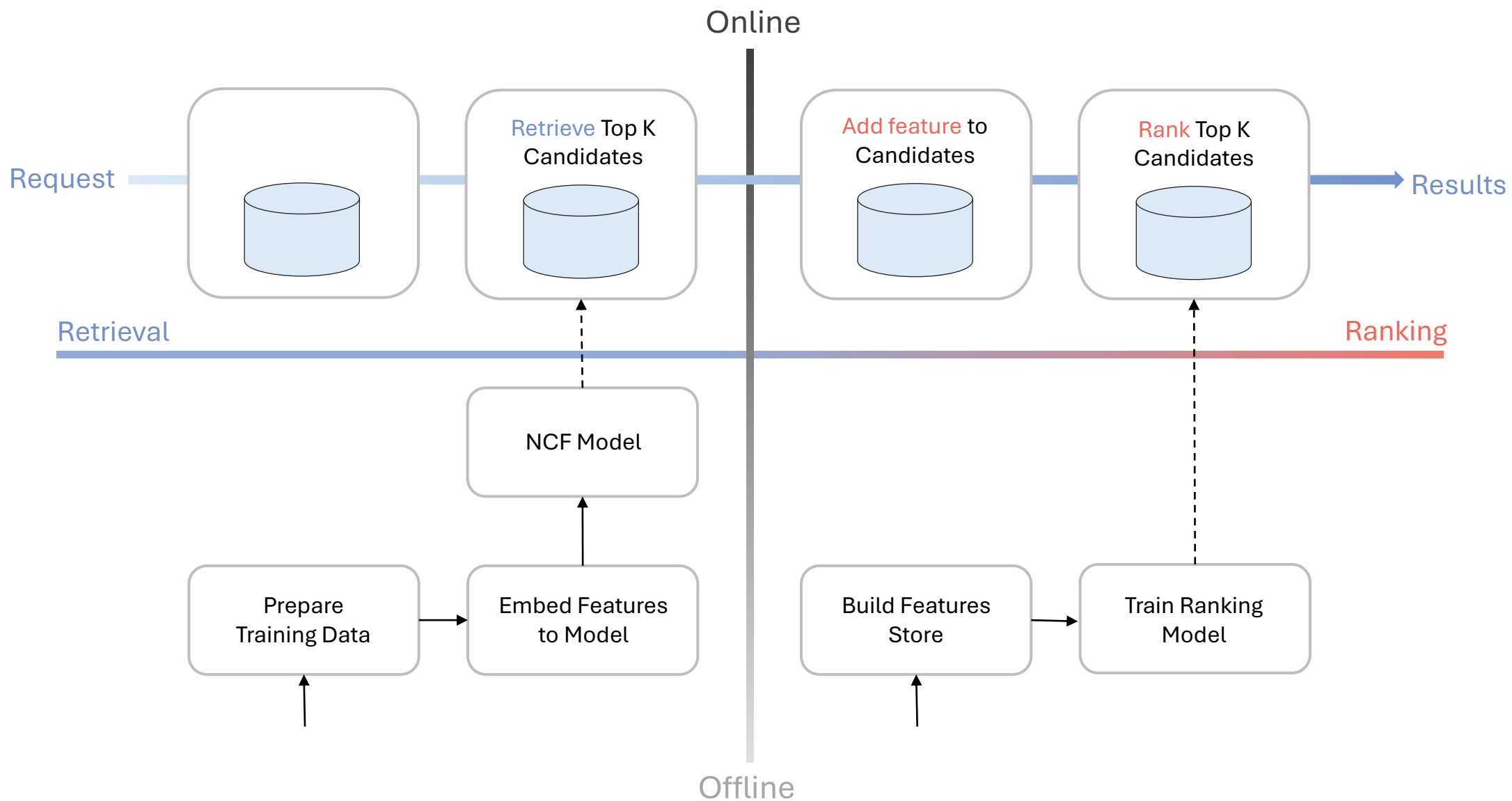


➤ GH Torrent Schema

	Period	Years
Training	2010 - 2015	6
Validation	2016 - 2018	3

Our data comes from GHTorrent, which was collected GitHub activity starting from 2007-10-29 but stopped updating after 2019-05-31.

The data is now stored on BigQuery *ghtorrentmysql1906.MySQL1906*



Statistics of the collected onboarding dataset

Dataset	GitHub
# of interaction data	350,000
# of developers	62,547
# of projects	49,125
# of interaction type	7 (each takes 50,000 samples)
Sparsity	99.9886%
# of community connection	

- Use 'user_fake' to delete fake users
 - **Test Account**
 - **Automatically Created Account**
 - **Data Retrieval Error**
- Use 'user_type'
 - **0** → **User** (Standard user account) **USR**
 - **1** → **Organization ORG**
(Organizational account: companies, OS teams)
 - choose to only analyze the 'user' account

- We analyzed six years of onboarding decisions (2010-2015) from GHTorrent
- Programming Language: eight popular general-purpose programming languages:
JavaScript, Java, Python, PHP, Ruby, C++, C, and C#
- Description: Delete projects with non-English and insufficient description(less than 5 words)
- Excluded projects that do not have at least 2 stars(watchers) and 5 developers.

In summary, we studied a total of 350,000 successful onboarding decisions associated with 62,547 developers and 49,125 open-source GitHub projects

Sparsity of the dataset is defined as follows: $1 - \frac{\text{Number of observed interaction}}{\text{Total possible interactions}}$



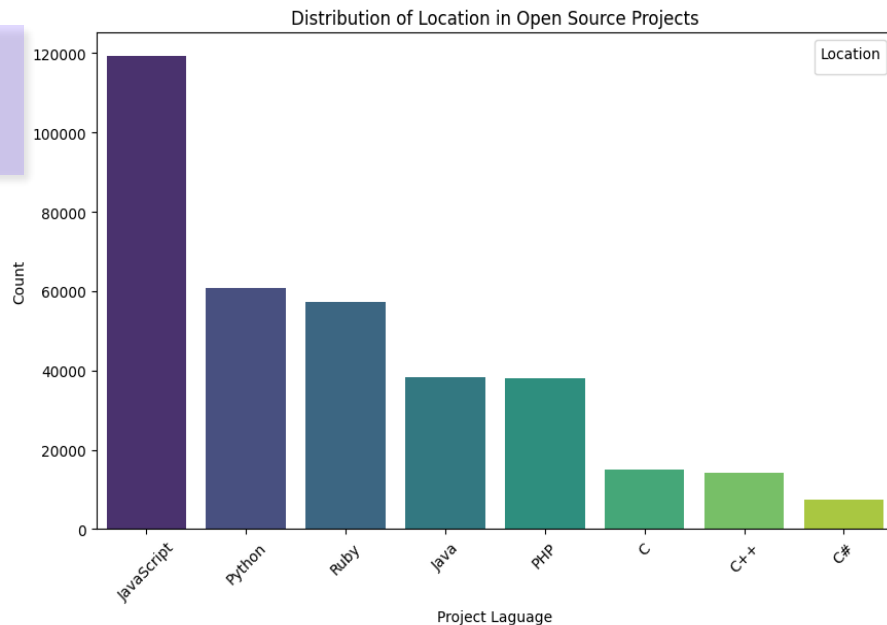
Distribution of Developer Interactions in the Dataset

We studied a total of 350,000 successful onboarding decisions associated with 62,547 developers and 49,125 open-source GitHub projects

Number of Interaction	Number of developer	Percentage
1	24963	39.91%
2	12502	19.99%
3	7458	11.92%
4	4793	7.66%
5	3231	5.17%
6	2211	3.53%
7	1571	2.51%
8	1160	1.85%
9	954	1.53%
>=10	3974	6.35%
Max	293	0.47%

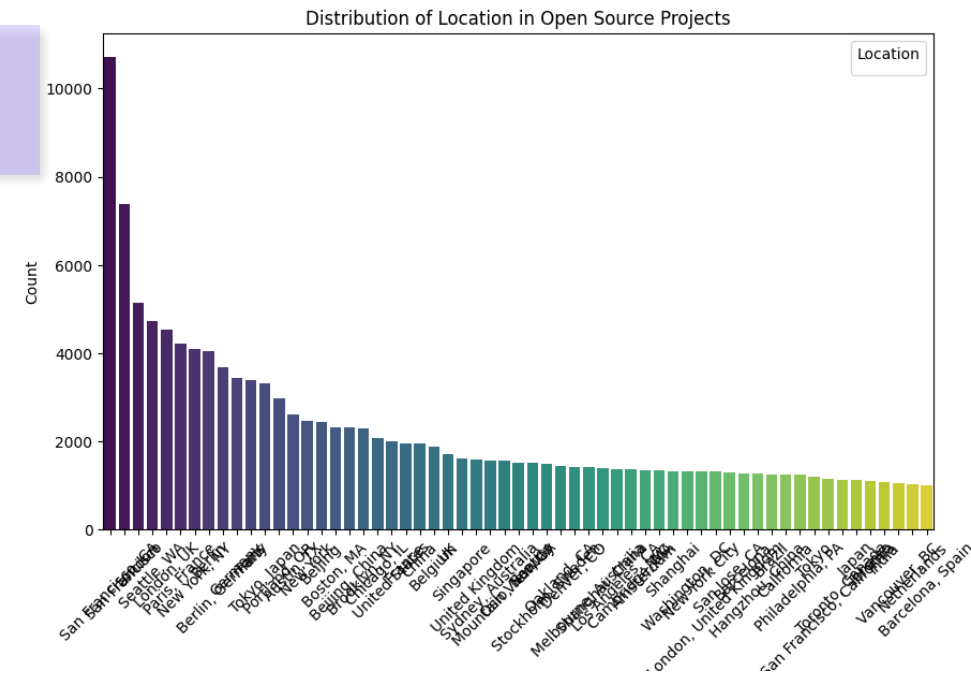
Project Language

Top 8



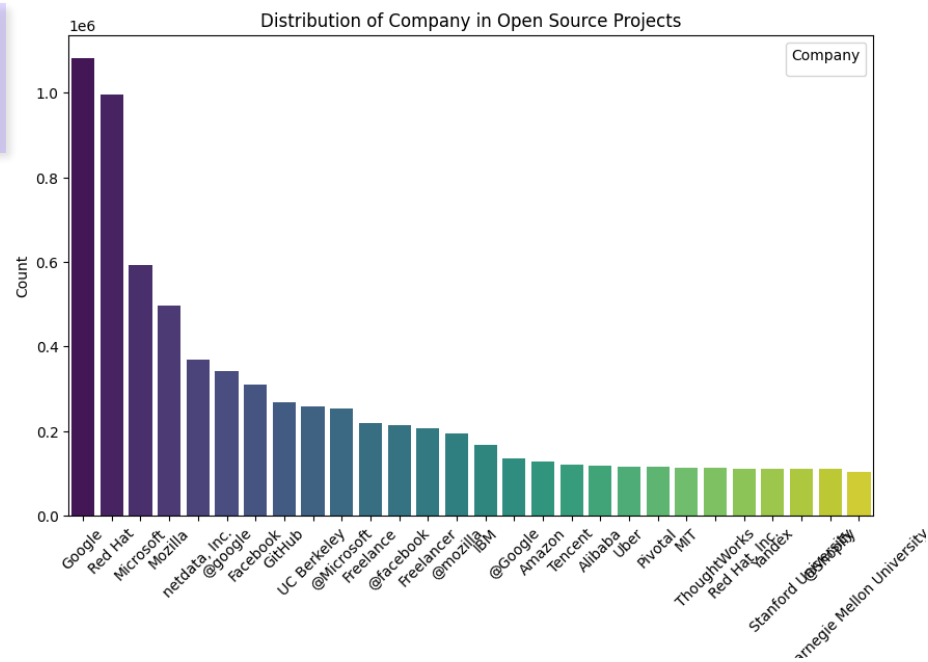
Location

8759



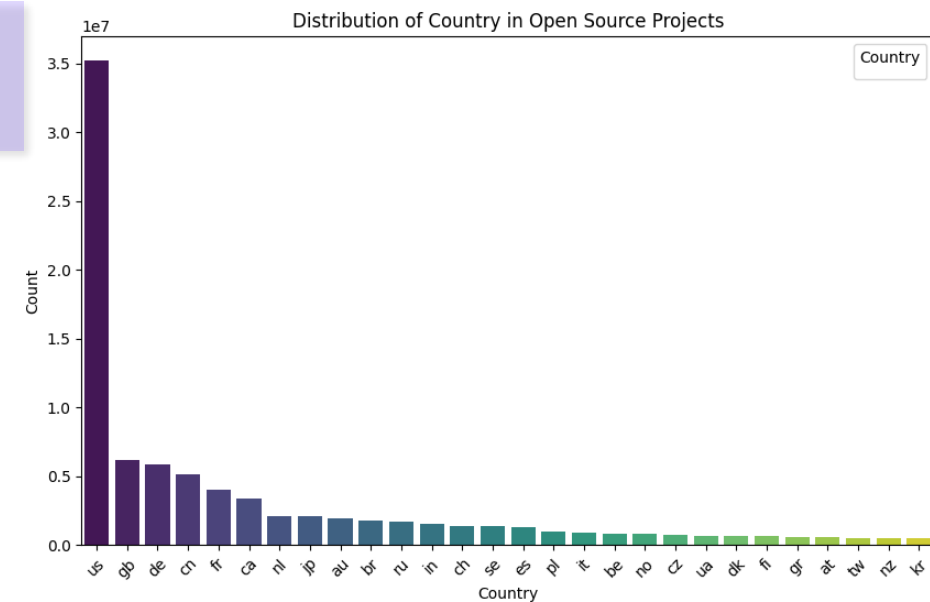
Company

44929



Country

162



Interaction Type

-  **STAR (Level 0):**
Indicates a light form of interest or appreciation for a repository, reflecting low-commitment engagement.
-  **FOLLOW (Level 1):**
Represents a user's intent to monitor another user's activity, implying a degree of trust or admiration.
-  **FORK (Level 2):**
Suggests a desire to explore or modify the repository further, signaling deeper engagement with the content.
-  **ISSUE (Level 3):**
Demonstrates active participation by raising concerns or feature requests, reflecting involvement in project development.
-  **PULL REQUEST (Level 4):**
Involves contributing actual code, denoting a strong level of engagement and willingness to collaborate.
-  **PULL REQUEST COMMENT (Level 5)**
Provides feedback on others' contributions, indicating strong interpersonal interaction and teamwork.
-  **COMMIT (Level 6):**
Represents the act of directly writing to the codebase, reflecting the highest level of participation and ownership.

Feature Sampling

Feature name	Definition
user_idx	Developer ID (index) – numerical ID for each developer
item_idx	Project ID (index) – numerical ID for each project
description	Project description vector – BERT pooled embedding
country	User’s country (index) – categorical feature, encoded as integer
location	User’s location (index) – city/state level, integer encoded
company	User’s company (index) – categorical (e.g., GitHub, Google)
language	User’s primary language (index) – categorical feature
profile_sim	Profile similarity score – numerical, log-transformed
social_tie	Social tie score – numerical, log-transformed
Technical_ability	Technical ability score - numerical
label	Interaction Rating – integer class from 0 to 6 (interaction type - rating)

Candidate Generation:
use clustering to let the
model learn the description
of each project

Tech Ability

- Developers are more likely to code in languages that they have used before

$$Tech_{lang} = T_i$$

Profile Similarity

- Developers prefer to join projects where **team members share similar backgrounds.**
(Company)
- Count how many **existing project members** share the same **company** as the developer.

$$Profile_{sim} = P_i$$

Social Tie

- Consider the projects: the developer and the project owner have previously collaborated

[3]

$$Social = \frac{1}{m_{it}}$$

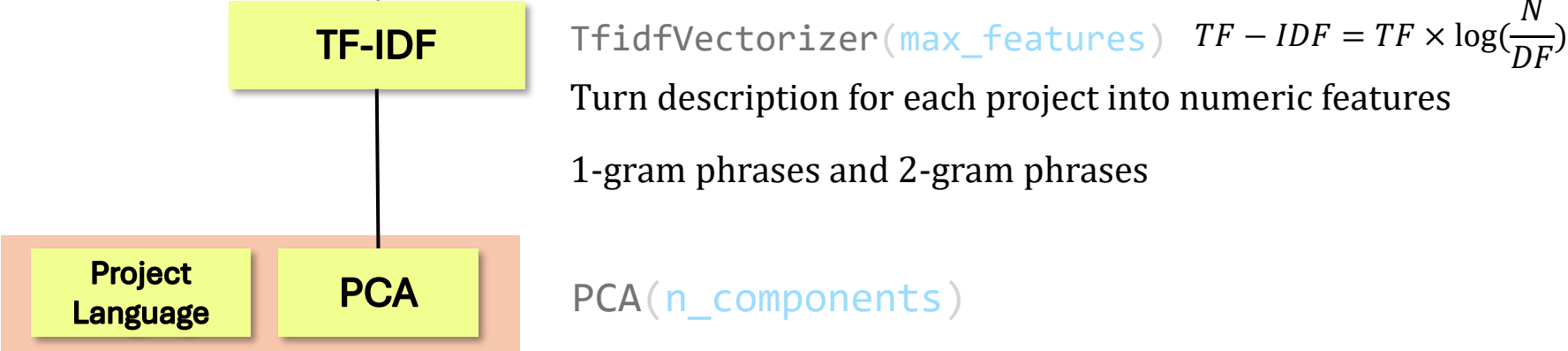
The fewer members in a past project
The stronger the social tie

number of member in each project(item)

Project-based Clustering

group similar Project into “buckets” (clusters).

	user	project	developer_id	project_id	interaction_type	interaction_time	project_owner_id	project_language	description	country_code
0	YoSoyVladman	django-social-auth	94758	1318	fork	2015-01-15 20:33:04+00:00	7518	Python	Django social authentication made simple	ca



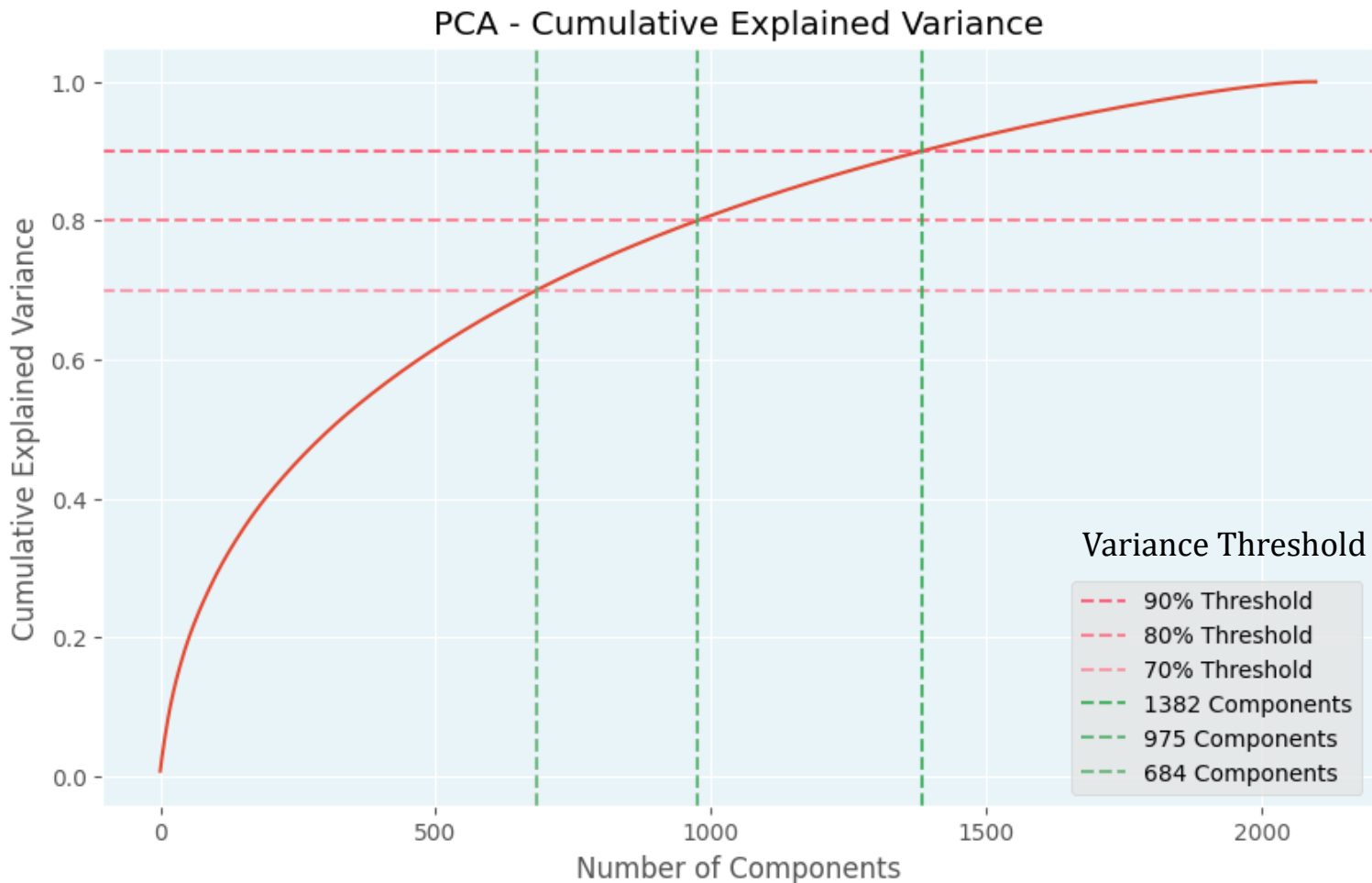
Total TF-IDF Tokens (features)	182,109
Total Samples (projects)	49125
TF-IDF Matrix size	(49125 × 182109)
Memory Needs	~ 66.7 GB

PCA- Principal Component Analysis

Determine the valuable number of PCA components

Total TF-IDF Tokens (features)	182,109
Total Samples (projects)	49125
TF-IDF Matrix size	(49125 × 182109)
Memory Needs	~ 66.7 GB

Max Feature	100	200	684	975	1382	5000	182,109
Sparsity	0.9683	0.9808	0.9924	0.9943	0.9945	0.9986	0.99974



Project-based Clustering

group similar Project into “buckets” (clusters).

	user	project	developer_id	project_id	interaction_type	interaction_time	project_owner_id	project_language	description	country_code
0	YoSoyVladman	django-social-auth	94758	1318	fork	2015-01-15 20:33:04+00:00	7518	Python	Django social authentication made simple	ca

TF-IDF

`TfidfVectorizer(max_features)` $TF - IDF = TF \times \log(\frac{N}{DF})$
Turn description for each project into numeric features

Project
Language

PCA

`PCA(975)`

Hierarchical

DBSCAN

K-mean

HDBSCAN

Create Project Cluster

Determine the optimal number of clusters

Evaluate Clustering Method

Silhouette Score

- How well each data point fits within its assigned cluster.
- How distinctly separated it is from other clusters.

$$\frac{b_i - a_i}{\max(a_i, b_i)}$$

- a_i : Intra – Cluster distance
- b_i : Nearest – Cluster distance
- +1
data point fits very well in its own cluster and is far from others.
- -1
data point is in the wrong cluster

Calinski-Harabasz Score

- How tight each cluster is, and how far apart the clusters are.

$$\frac{B}{W} \times \frac{N - K}{K - 1}$$

- A higher score is better clusters are tight and well-separated. It helps determine the ideal number of clusters.

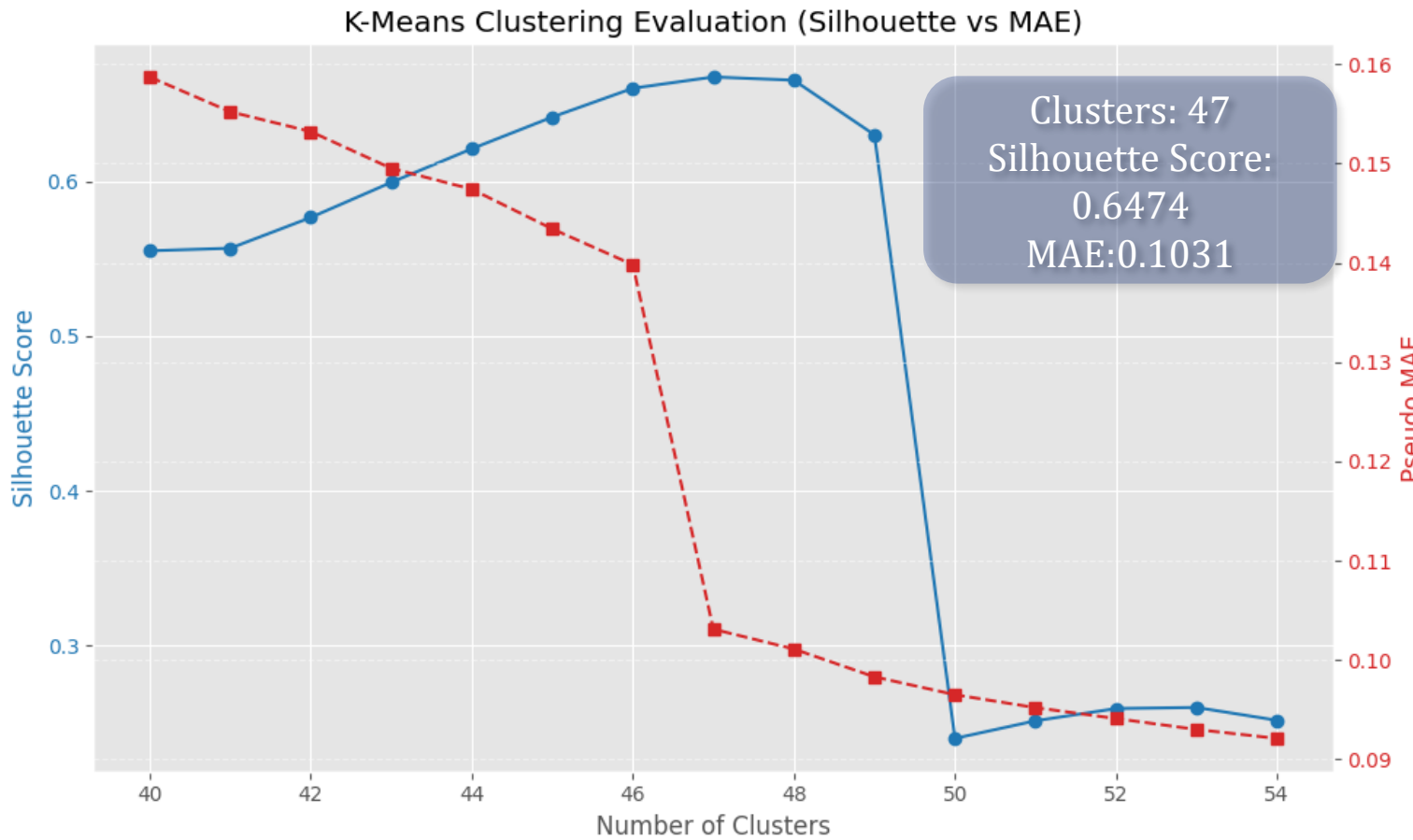
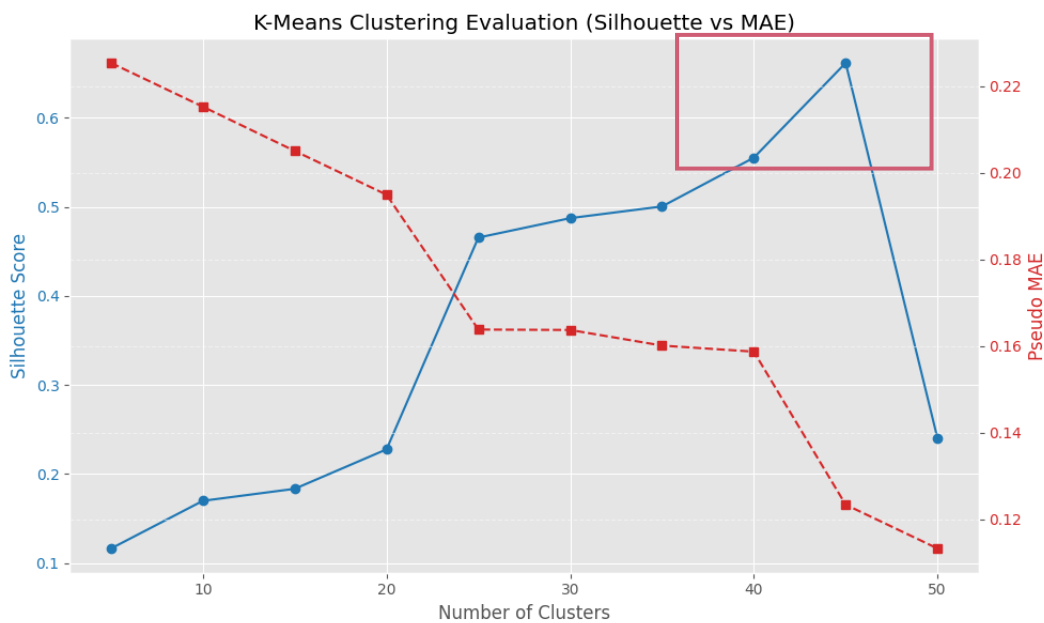
Davies-Bouldin Index

- How tight each cluster is, and how far apart the clusters are.

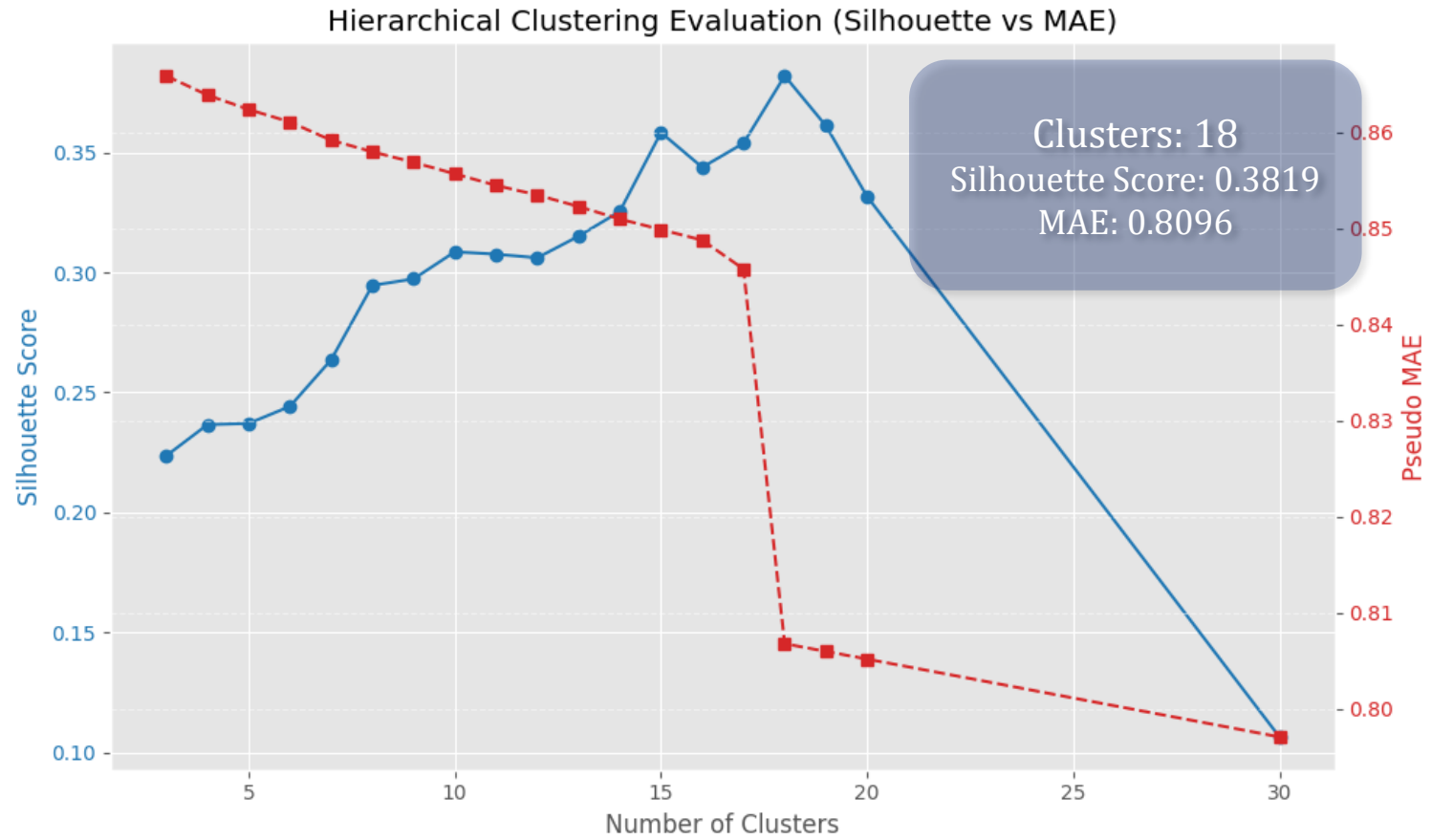
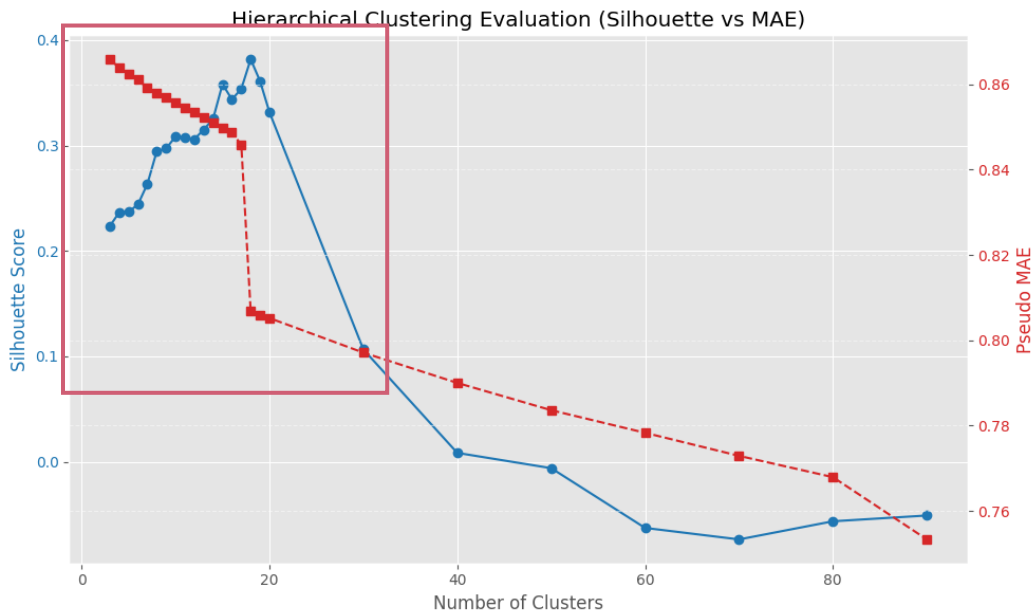
$$\frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{R_{ii} + R_{jj}}{R_{ij}} \right)$$

- Lower DBI: Better, Clear clusters
- Higher DBI: Messy, Overlapping

Clustering – K-Means



Clustering – Hierarchical Clustering

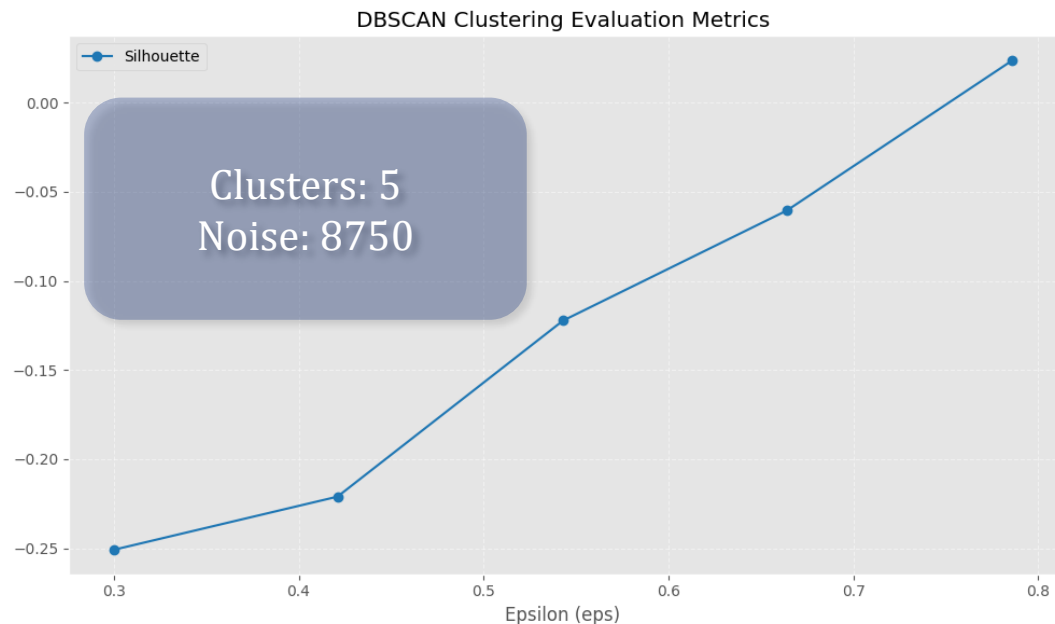


Clustering – DBSCAN

- The **best Silhouette Score achieved was only 0.0236**
- Significant data labeled as noise

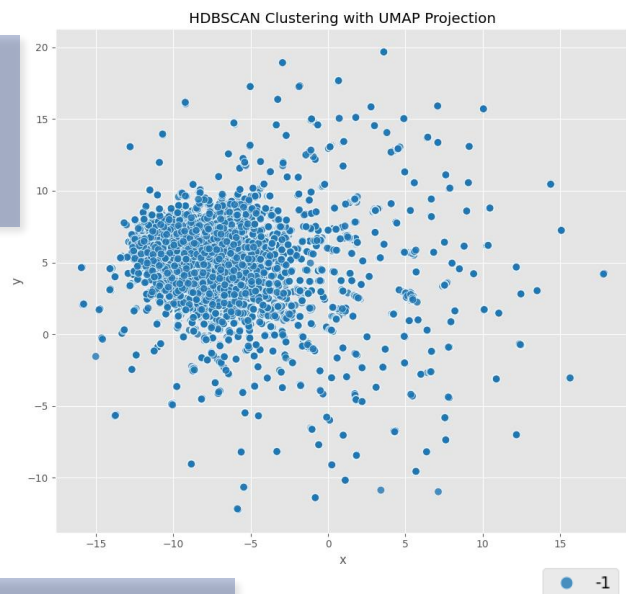
 **Conclusion:** DBSCAN proved ineffective in this context

min_sample = 15

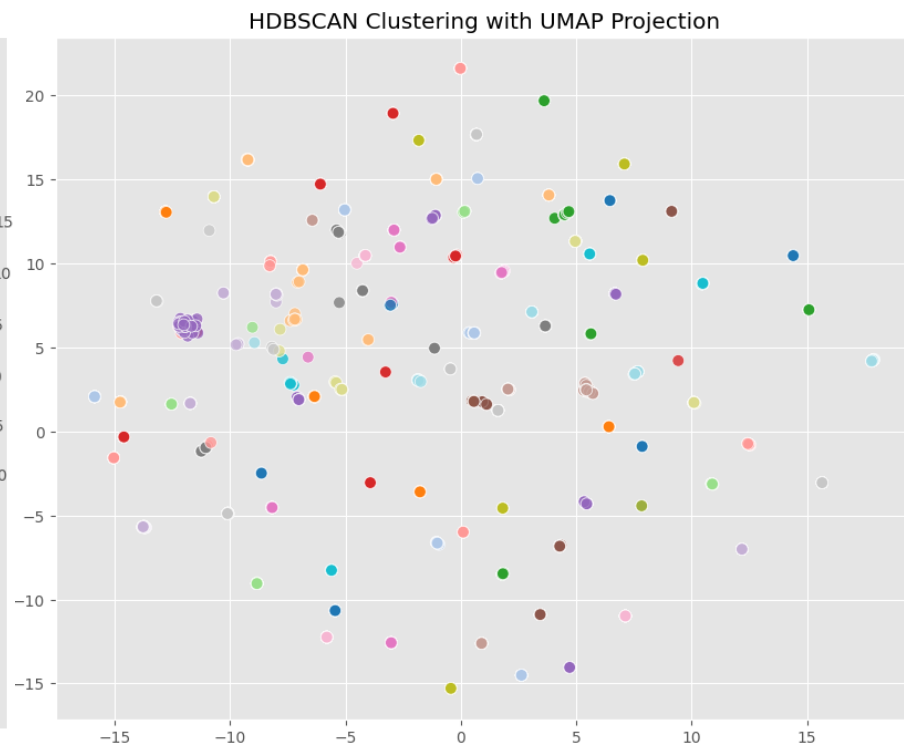
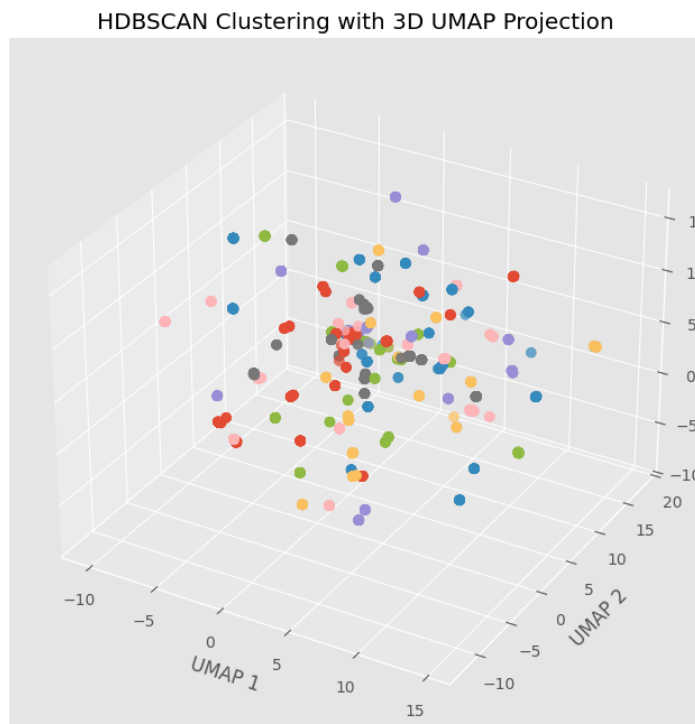


Clustering – HDBSCAN

89 Cluster



Silhouette Score: 0.1808

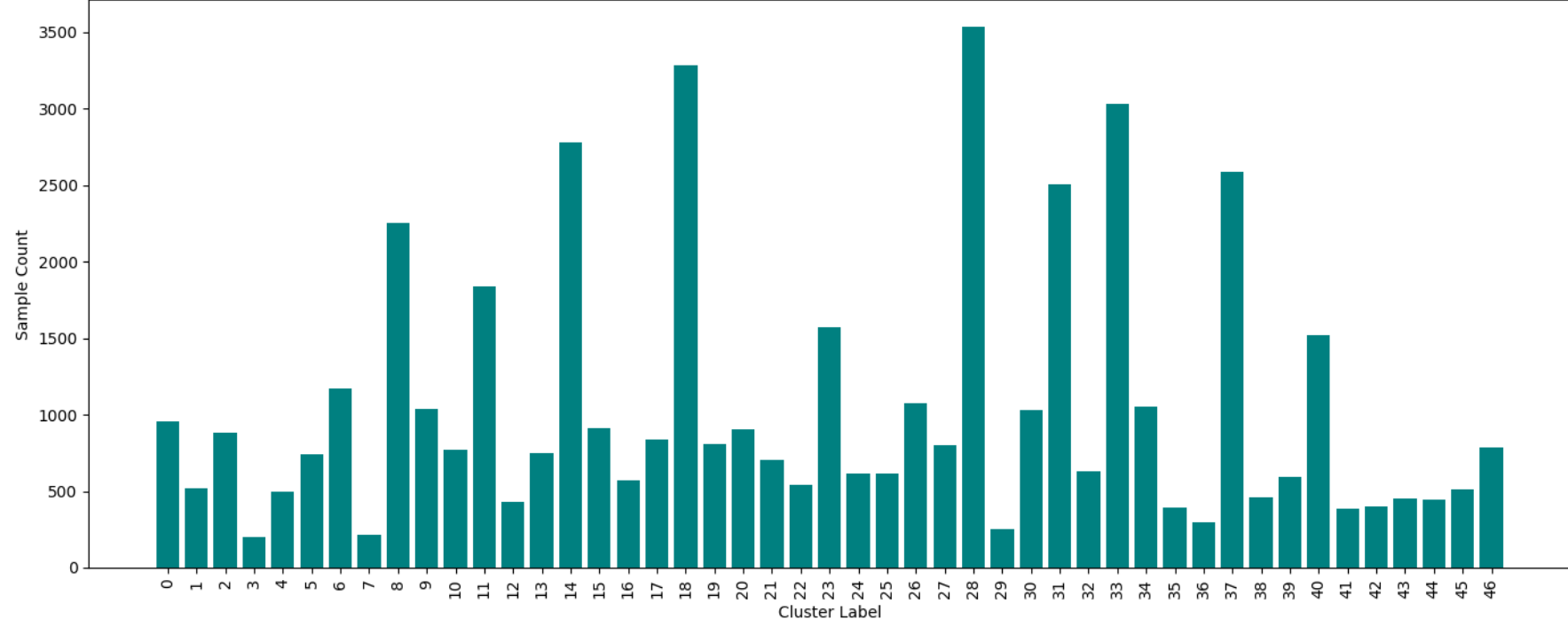
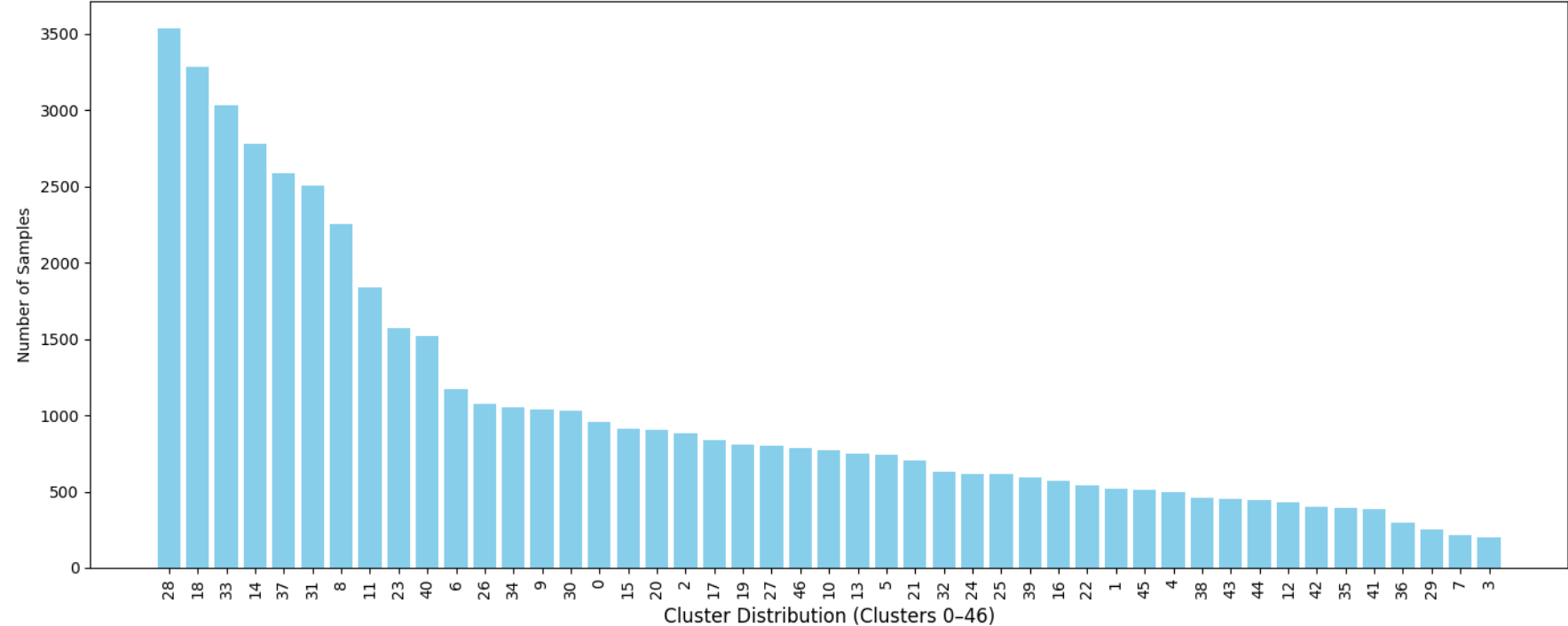


Unsupervised Topic Modeling

	#Cluster	Davies-Bouldin Index	Calinski-Harabasz Score	Silhouette Score
K-means	47	0.9690	28.2601	0.6474
K-means	522	1.4632	15.2942	0.2294
Hierarchical Clustering	18	1.2344	18.5077	0.3819
DBSCAN (Eps = 0.786)	5	13.7897	16.1000	0.0236
HDBSCAN	89	2.1008	9.0208	0.1808

Comparison of Clustering Algorithms Using Unsupervised Metrics with TF-IDF and PCA for Topic Modeling

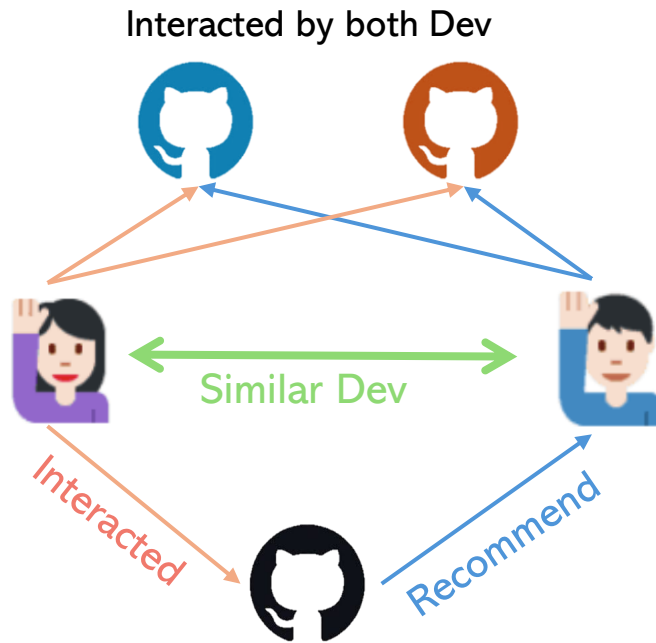
Cluster Distribution



Cluster	Top Keyworkds	Cluster	Top Keywords
0	plugin, event driven, driven, android, based, tracker, event tracker, using, library, custom	24	official, git, git repository, main, code, project, mirror, main repository, official repository, code repository
1	web, repository, framework, web development, environment, development environment, development repository, software, project, application	25	javascript, written, java, language, compiler written, python, php, javascript compiler, using, library
2	management, platform, business logic, logic, intelligence, business intelligence, small business, small, process, web	26	chef, chef cookbook, repository, development, development repository, repository chef, opscore, opscore cookbook, repository opscore, cookbook install
3	official, based, simple, using, view, track, create, displays, ios, version	27	final, final project, django, data, management, tool, based, django project, using, repo
4	interface, line interface, tool, line tool, client, tools, line tools, utility, line client, line utility	28	library, simple, tool, based, using, module, management, rails, use, tools
5	pull, pull requests, http, http requests, library, github, repository, using, mirror, based	29	incubating, apache cordova, cordova, flink, apache flink, plugin, cordova plugin, apache hadoop, hadoop, apache maven
6	jquery, wordpress, jquery plugin, wordpress plugin, plugin wordpress, simple, text, plugin jquery, grunt, using	30	php, javascript, php framework, based, application, data, building, python, framework building, javascript framework
7	website, repository, project, code repository, android, framework, tool, site, library, documentation	31	api, php, library, client, wrapper, simple, rest, rest api, php library, data
8	library, python library, api, client, wrapper, implementation, python wrapper, python client, interface, simple	32	performance, distributed, high, high performance, data, based, database, framework, library, applications
9	app, web app, interface, applications, web applications, web interface, api, client, based, services	33	rails, android, rails app, django, django app, app android, mobile, google, simple, engine
10	library, application, android library, client, android application, android client, simple, library android, view, client android	34	api, rails, ruby rails, library, wrapper, ruby wrapper, gem, client, ruby gem, ruby client
11	client, library, api, module, javascript, nodejs client, browser, simple, library nodejs, client nodejs	35	javascript, python, library, ruby, generator, written, based, php, json, fast
12	engine, game engine, based, javascript, multiplayer, html, using, card, simple, project	36	html, javascript, html css, js, plugin, framework, using, css javascript, sass, files
13	like, library, data, api, project, plugin, designed, library provide, way, aims	37	library, client, api, java library, implementation, framework, java client, library java, written, simple
14	javascript, data, library, javascript library, implementation, client, tool, simple, analysis, using	38	library, adds, plugin, django, python, android, web, module, provides, extension
15	server, mirror, code, repository, based, thesis, template, project template	39	component, component zend, module, framework module, doctrine, module doctrine, repository, git repository, git, framework git
16	dropin, python, dropin replacement, android, support, module, provides, standard, based, classes	40	web, web application, django, rails, data, framework, management, simple, using, application framework
17	android, api, php, android sdk, javascript, nodejs, php sdk, azure, javascript sdk, python	41	chrome, chrome extension, yii, php, google, typo, google chrome, spree, web, typo extension
18	management, free, platform, data, open, engine, software, server, web, framework, api	42	framework, mvc framework, aspnet, aspnet mvc, javascript, php, nodejs, web, based, simple
19	api, client, web, nodejs, application, framework, simple, http, web server, client server	43	programming language, ruby, ruby programming, web, new, clojure, clojure programming, compiler, python, javascript
20	Use, easy use, library, way, easy way, makes, make, android, makes easy, nodejs	44	wordpress, wordpress theme, bootstrap, theme wordpress, based, website, framework, starter, responsive, starter theme
21	python, project, tool, javascript, ruby, python code, used, using, api, data	45	framework perfectionists, perfectionists deadlines, deadlines, perfectionists, nodejs, php, framework nodejs, django, simple, django web
22	personal, personal website, new, code, source, project, repository, static, blog, official		
23	open, web, management, cross, cross platform, data, management platform, based, cloud, application	46	python, ruby, python bindings, ruby bindings, nodejs, api, library, bindings nodejs, node, javascript

Neural Collaborative Filtering

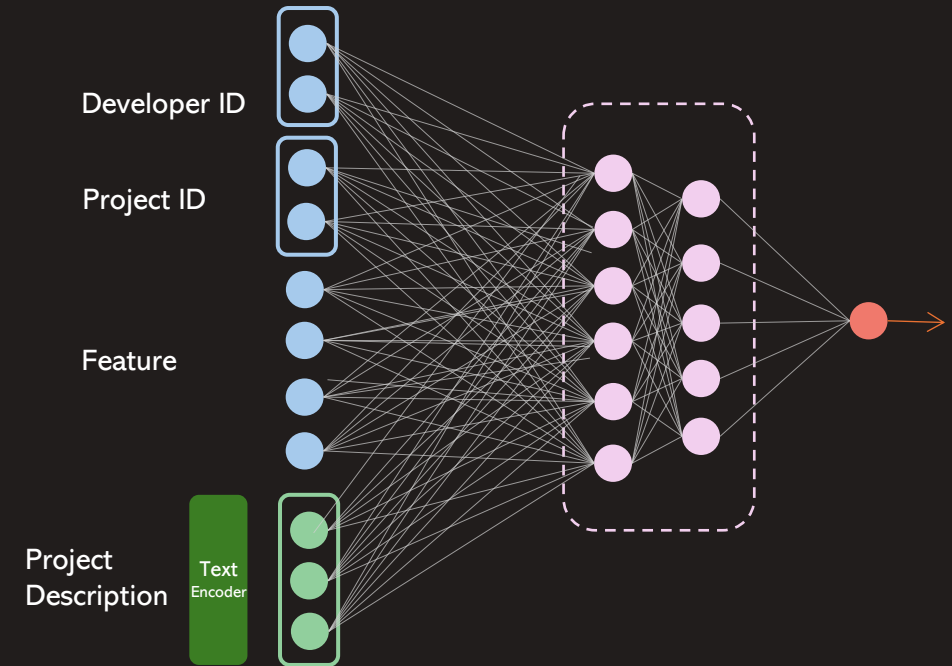
Model the similarities between developers and projects *simultaneously* to generate personalized recommendations.



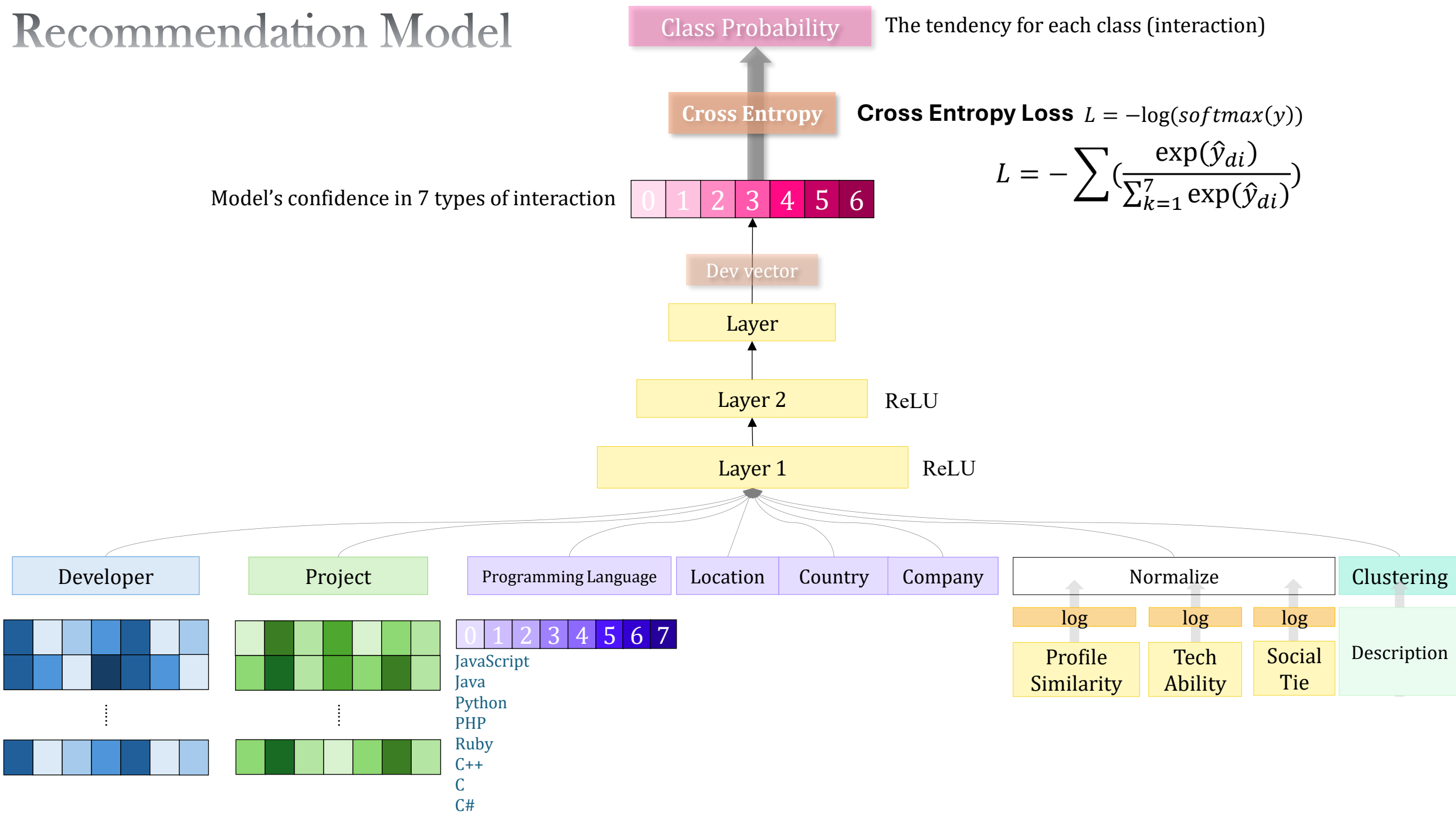
Sparse Features

Dense Features

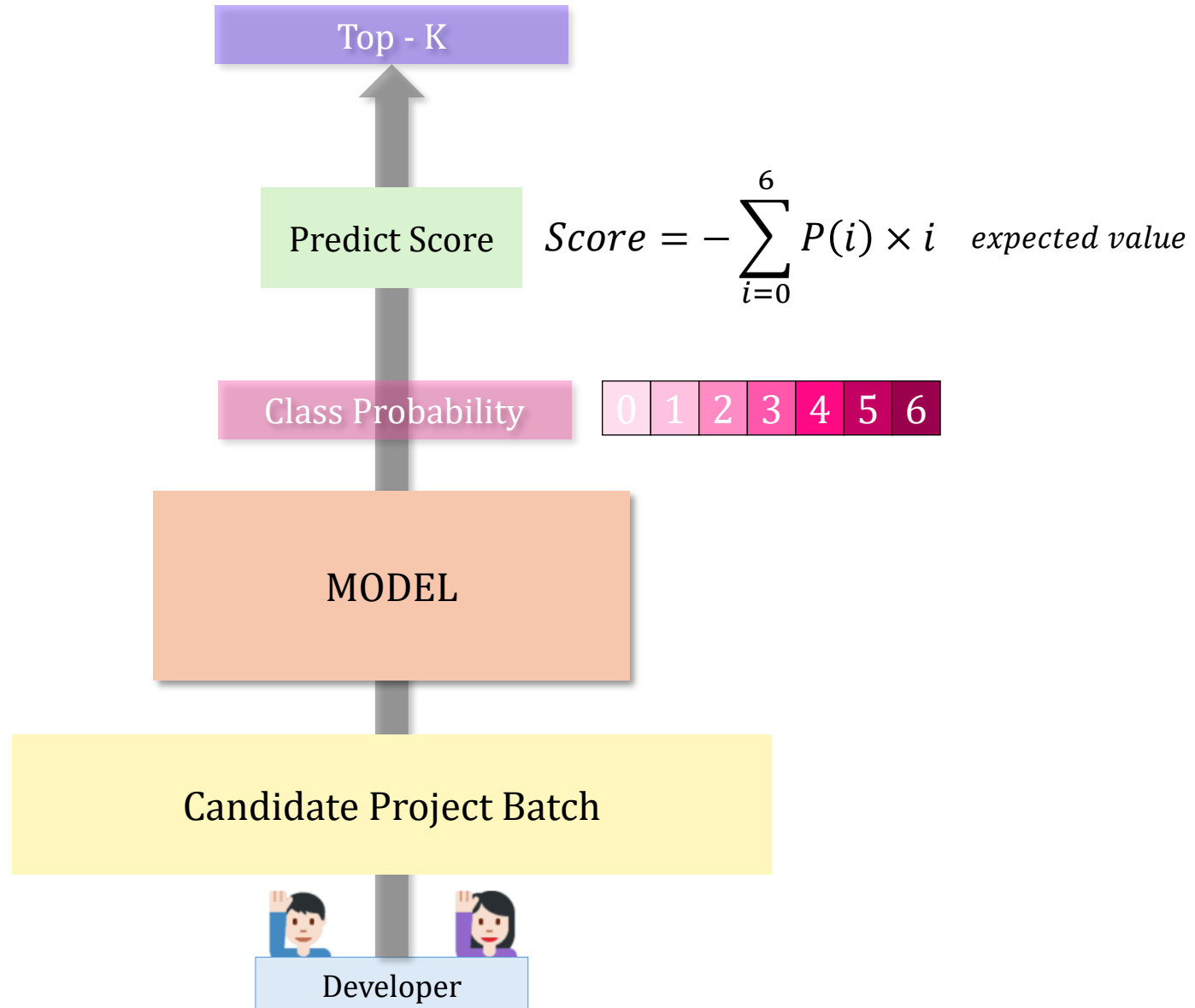
If user A is similar to user B, and user B likes project 1, then the system can recommend project 1 to user A (even if user A hasn't seen any project similar to project 1).



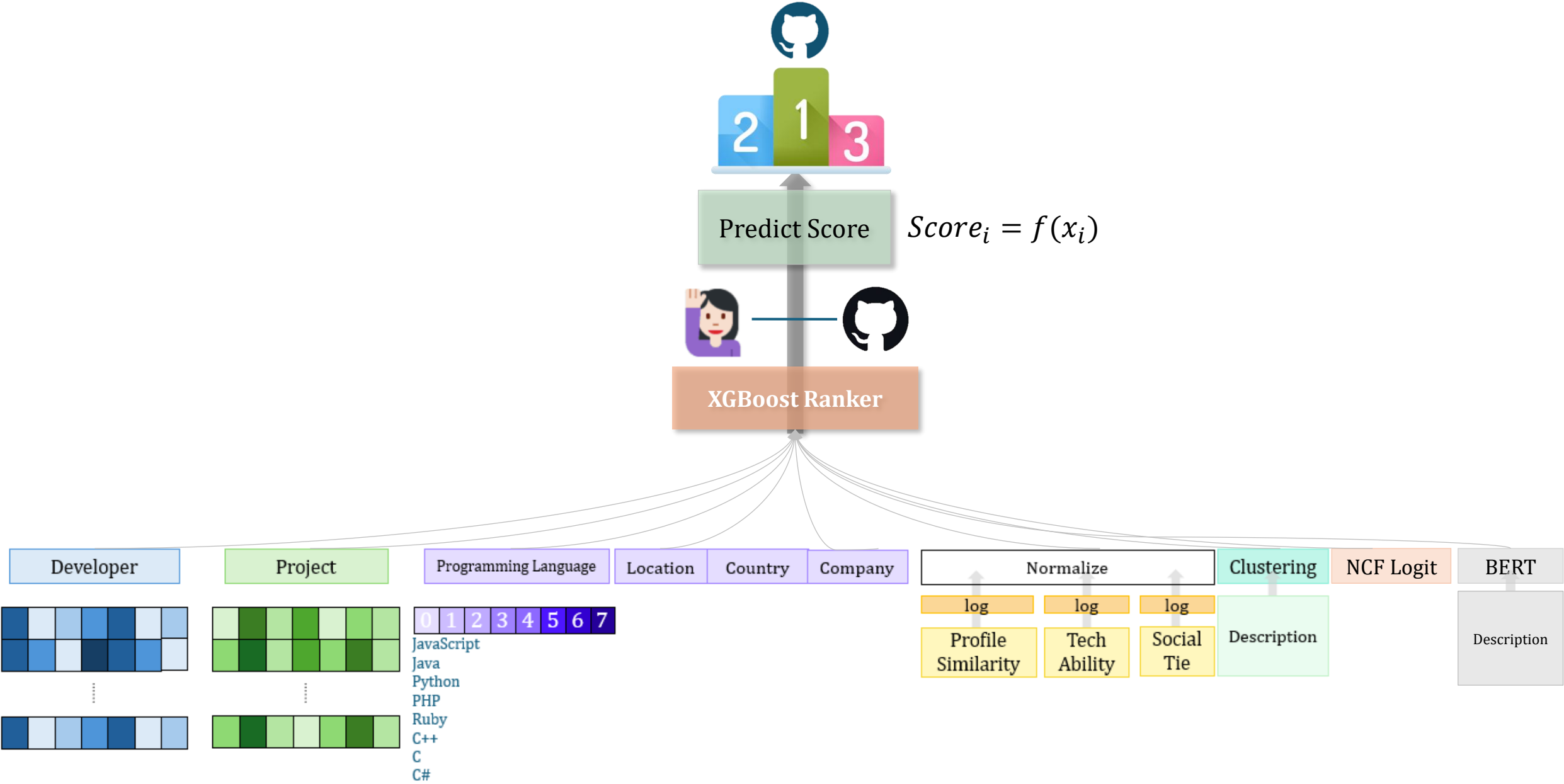
Recommendation Model



Generate Recommendation



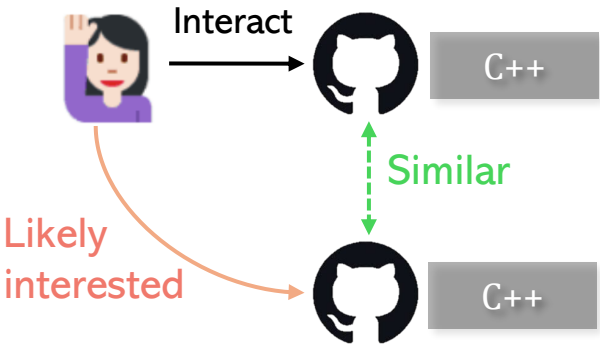
Ranking - XGBoosting ranking



Baseline Model Selection

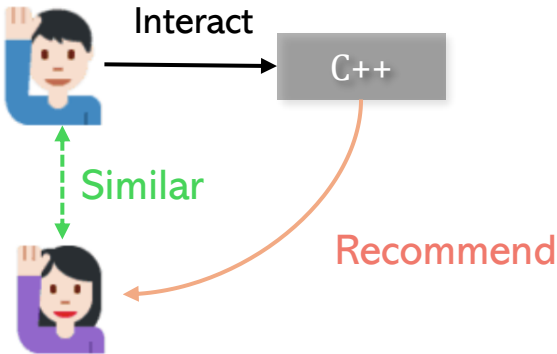
Project-Based

The Recommender system suggests a new project based on the project's similarity to projects previously selected by the user

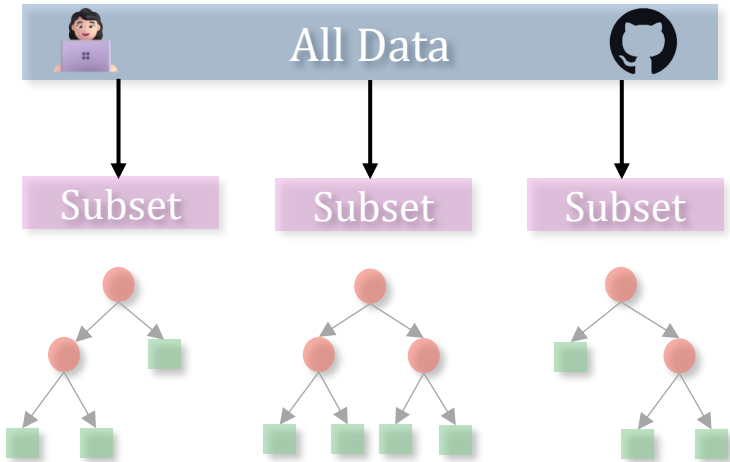


Developer-Based

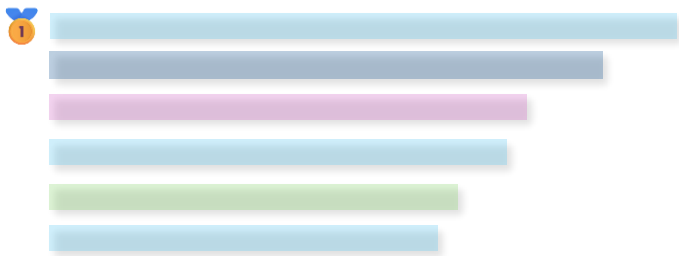
Recommends items similar to those the developer interacted with.



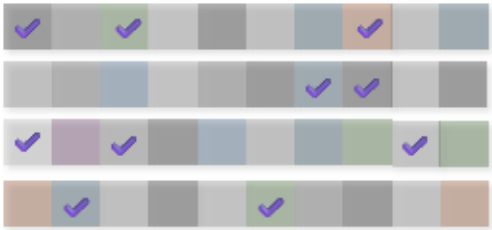
XGBoost



Top 10 Popular



Random



Evaluation Metrics

Precision @ K

$$\text{Precision} = \frac{\text{Number of Relevant items in Top } K}{K}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall @ K

$$\text{Recall} = \frac{\text{Number of Relevant items in Top } K}{\text{Total number of relevant items}}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

MAP @ K

The average precision across different relevant items and users, placing more weight on **ranking correctly**.

Mean Average Precision @ K

NDCG

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}$$

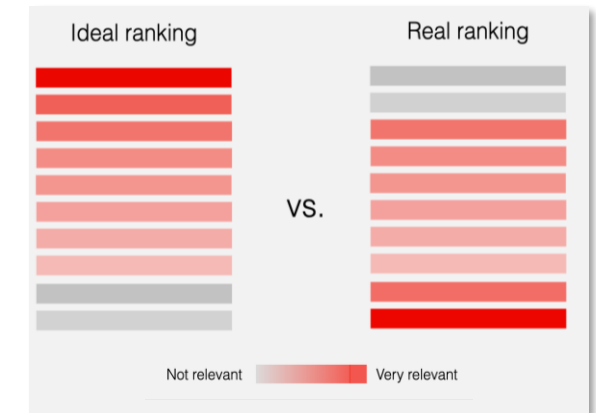
Normalized Discounted Cumulative Gain

Discounted Cumulative Gain

$$\text{DCG} = \sum_{i=1}^P \frac{rel_i}{\log_2(i+1)}$$

Ideal Discounted Cumulative Gain

$$\text{IDCG} = \sum_{i=1}^P \frac{rel_i^*}{\log_2(i+1)}$$



NDCG lies from 0 to 1, with an ideal rank represented at 1
 p shows the rank status. rel_i shows the relevance score of the item at position i
IDCG is calculated the same as DCG except that it uses the ideal ranking order.

Evaluation

NCF model

	Recall			
	@10	@50	@100	@300
Our Model (K means 47)	0.02931	0.05130	0.14657	0.3139
Our Model (H 18)	0.01975	0.03037	0.08262	0.1826
XGBoost	0.01834	0.02463	0.07556	0.1198
Use-based kNN	0.00957	0.01082	0.06866	0.0964
Item-based kNN	0.01832	0.02769	0.08146	0.1059
Random	0.00036	0.00051	0.00086	0.0011
Top 10	0.00006	0.00018	0.00033	0.00034

NCF + XGBoosting

	Recall @10	Recall @ 50
Our Model (K means 47)	0.17591	0.26793
Our Model (H 18)	0.08294	0.18533
XGBoost	0.01834	0.02463
Use-based kNN	0.00957	0.01082
Item-based kNN	0.01832	0.02769
Random	0.00036	0.00051
Top 10	0.00006	0.00018

Evaluation – NCF model

	Recall				NDCG			
	@10	@50	@100	@300	@10	@50	@100	@300
Our Model (K means 47)	0.02931	0.05130	0.14657	0.3139	0.03372	0.03547	0.03985	0.04925
Our Model (H 18)	0.01975	0.03037	0.08262	0.1826	0.02624	0.02788	0.02892	0.03352
XGBoost	0.01834	0.02463	0.07556	0.1198	0.02447	0.02740	0.02943	0.03422
Use-based kNN	0.01832	0.02769	0.08146	0.1059	0.01933	0.02469	0.02736	0.03104
Item-based kNN	0.00957	0.01082	0.06866	0.0964	0.01486	0.01744	0.02536	0.03073
Random	0.00036	0.00051	0.00086	0.00117	0.00067	0.00082	0.00109	0.00198
Top 10	0.00006	0.00018	0.00033	0.00034	0.00000	0.00000	0.00187	0.00236

Evaluation – NCF + XGBoosting

	Recall @10	Recall @ 50
Our Model (K means 47)	0.17591	0.26793
Our Model (H 18)	0.08294	0.18533
XGBoost	0.01834	0.02463
Use-based kNN	0.00957	0.01082
Item-based kNN	0.01832	0.02769
Random	0.00036	0.00051
Top 10	0.00006	0.00018

	NDCG @10	NDCG @ 50
Our Model (K means 47)	0.06294	0.08522
Our Model (H 18)	0.05842	0.06041
XGBoost	0.02447	0.02740
Use-based kNN	0.01933	0.02469
Item-based kNN	0.01486	0.01744
Random	0.00067	0.00082
Top 10	0.00000	0.00000



Thank You

For listening

Jasmine Yang
Chintan Armit